Why don't we use HTTP module Directly ?

- verbose and repetitive
- No routing . middleware , or extra features
- Hard to manage Large Application

Introduction to EXPRESS :

- Too simple to create and manage Servers
- lightweight and consume less system contents

```
import express from 'express'
// or we can  use {const express = require('express')}

const app = express()

app.get('/', (req, res) => {
res.send('Hello World')
})

app.listen(3000,()=>{
    console.log(`Server is running on 3000`));
}
```

🔍 Line-by-Line Explanation:

`import express from 'express'`

- **Purpose:** This imports the `express` **package/module**.
- `express` is a minimal, flexible Node.js web framework used to build APIs and web apps.
- `import ... from` is **ES6 module syntax**, often used instead of `require()` .

> ✅ Make sure `"type": "module"` is set in `package.json` to use `import` .

`const app = express()`

- You are **calling the Express function**, which returns an **application instance**.
- `app` now represents your **server application**.
- This instance has methods like `.get()`, `.listen()`, etc.

`app.get('/', (req, res) => { ... })`

✅ `app.get(path, callback)`

- This defines a **GET route** on your server.
- `GET` is an HTTP method (like POST, PUT, DELETE).

✅ `'/'`

- This is the **URL path**. `'/'` refers to the **home page** (`http://localhost:3000/`).

✅ `(req, res) => { ... }`

- This is the **callback function** that runs when someone visits the route.
- It receives two arguments:
  - `req` → the **request object** (contains data about the incoming request).
  - `res` → the **response object** (used to send a response back to the client).

✅ `res.send('Hello World')`

- Sends the string `"Hello World"` as a **response** to the client.
- Automatically sets status code `200 OK`.

`app.listen(3000, () => { ... })`

✅ `app.listen(port, callback)`

- This starts your **server** and tells it to listen for requests on port `3000`.

✅ `3000`

- The **port number** on your local machine.

✅ `() => { console.log(...) }`

- A **callback function** that runs **once the server starts successfully**.

✅ Output:

When you run this server and go to `http://localhost:3000/` in your browser, you'll see:

```
Hello World
```

And in the terminal, you'll see:

```
Server is running on 3000
```

🧠 Bonus: What's Happening Behind the Scenes?

- The Express `app` acts like a **traffic controller**.
- When a browser makes a request (like GET `/`), Express **routes it** to the correct handler (`app.get`).
- You define what to do for each route.
- Express wraps the raw HTTP server from Node.js, making routing and handling much simpler.

📌 Summary Table of Key Parts:

Code	Purpose
`import express from 'express'`	Import Express framework
`const app = express()`	Create an app instance
`app.get('/', handler)`	Define a GET route for `'/'`
`(req, res)`	Request and response objects

Code	Purpose
`res.send('Hello World')`	Send response to client
`app.listen(3000, callback)`	Start server and listen on port 3000

What is an API ?

An API (Application Programming Interface) is a set of rules and protocols that enables different software applications to communicate, exchange data, and interact with each other. APIs act as intermediaries or "bridges" between systems, allowing developers to use existing features and data from other software without needing to understand or access their internal workings

APIs come in several types, each designed for specific use cases and communication styles. The main types of APIs and their differences are as follows:

API Type	Description	Key Differences
REST API	Uses HTTP methods (GET, POST, PUT, DELETE) and typically exchanges data in JSON or XML format.	Stateless, easy to use, widely adopted for web services, scalable, and flexible[2][6].
SOAP API	Relies on XML-based messages and strict standards for communication over various protocols.	More rigid and secure, supports complex operations, often used in enterprise and financial systems[2][3][4].
GraphQL API	Allows clients to specify exactly what data they need, reducing over-fetching.	More flexible queries, efficient data retrieval, but requires more setup and learning curve[1][6].
gRPC API	Uses protocol buffers for data serialization and HTTP/2 for	High performance, suited for microservices, supports multiple

API Type	Description	Key Differences
	transport, enabling fast communication.	languages, but more complex to implement[1][3][6].
RPC API	Executes remote procedures (functions) on a server, returning results or errors.	Focuses on actions rather than resources, can use JSON or XML, suitable for direct function calls[1][4].
WebSocket API	Enables real-time, two-way communication between client and server.	Ideal for applications needing instant updates (e.g., chat apps, live feeds), maintains open connections[5][6].
Composite API	Combines multiple API calls into a single request.	Useful for complex operations requiring data from several sources in one call[1].

Why they are different:

- **Communication Style:** REST is resource-oriented, RPC is action-oriented, GraphQL is query-based, and SOAP is document/message-based.
- **Data Format:** REST and GraphQL often use JSON, SOAP uses XML, gRPC uses protocol buffers.
- **Complexity and Flexibility:** REST is simple and flexible, SOAP is strict and robust, GraphQL is highly customizable, gRPC is optimized for speed and efficiency.
- **Use Cases:** REST is common for web/mobile apps, SOAP for enterprise systems, GraphQL for complex data needs, gRPC for microservices, and WebSocket for real-time apps.